ELL7301 / ELL734 -- Digital CMOS VLSI Design

Assignment 2

8-bit MAC Unit on SKY130 PDK RTL Design and Functional Verification

A report submitted in partial fulfilment of the requirements
for the award of the degree of
Master of Technology
in
Instrument Technology

by
BETHA NAVYA
Entry No. 2025JID2497

under the supervision of
Prof. Ankesh Jain



INDIAN INSTITUTE OF TECHNOLOGY DELHI
Hauz Khas,110016

Table of Contents

Part1:1.Introduction 4
2. RTL Architecture 4
2.1 Module: mac8.v 4
3. Functional Verification 4
3.1 Test plan and coverage 5
3.2 Testbench: mac8_tb.v 5
3.3 Simulation results 6
3.4 Waveform Evidence 7
4. Part 1 Summary 8
Part 2: Baseline Synthesis and Timing Analysis 9
2.1 Baseline Architecture 9
2.2 Synthesis Methodology 9
2.2.1 Synthesis TCL script (synth_baseline.ys) 9
2.3 Baseline Synthesis Results 9
2.4 Timing Constraints and Analysis Methodology 10
2.4.1 Timing constraints (constraints/constraints.sdc) 10
2.4.2 OpenSTA commands 10
2.5 Baseline Timing Results 10
Part 2(a) -- Does the baseline meet 250 MHz? 11
Part 2(b) -- Which block dominates the critical path? 11
Part 2(c) -- Architecture modification 11
Part 3: Architecture Optimization -- Pipelining 11
3.1 Pipeline Architecture Background 11
3.2 Design Space Explored 12
3.2.0 Visual Overview: Area and Timing Across All Iterations 12
3.2.1 Iteration 1: 2-stage naive pipeline (multiply | accumulate) 13
3.2.2 Iteration 2: 3-stage carry-save pipeline 15
3.2.3 Iteration 3: 4-stage pipeline (resolve adder split 10+10) 18
3.2.4 Iteration 4 (final): 5-stage pipeline (resolve adder split 5+5+10) 20
3.3 Answers to Assignment Questions 23
Question 3(a): Why does pipelining reduce critical-path delay, and why does it generally increase area and clock power? 23
Question 3(b): Does increasing the number of pipeline stages always improve the design? (timing, area, power, latency) 24
Part 4: Physical Implementation (RTL-to-GDSII) 24
4.1 Methodology 24
4.1.1 OpenLane configuration (config.json) 25
4.2 Final Architecture Carried Into Physical Implementation 25

4.3 Physical Design Flow: Iteration History **27**
4.4 Final Signoff Results (Run 3, mac8_pipelined6) **27**
4.4.1 Physical verification evidence **28**
4.4.2 Note on the max-fanout warning **28**
Part 5: Final PPA Summary Against Assignment Constraints **28**
5.1 Why Area Exceeds Target -- Honest Analysis **29**
Item 6: Throughput Investigation **29**
Item 4(f) / 8(f): Physical Design Stages **29**
i. Floorplan **30**
ii. Placement **30**
iii. Clock Tree **31**
iv. Routing **31**
v. Final Layout (GDSII) **32**
Item 9: Setup and Hold Timing Discussion **32**
Item 12: Summary of Final Architecture and Results **33**
Item 13: Final Summary Table **33**
Item 7: Project Directory Structure **34**
Overall Report Summary **34**

Part1:

1. Introduction

This document covers Part 1 of the assignment: the RTL design of a synchronous 8-bit x 8-bit multiply-accumulate (MAC) unit and its self-checking verification testbench. The MAC computes Y(n+1) = Y(n) + A(n) x B(n) every clock cycle when enabled, with a synchronous active-high reset. All arithmetic is unsigned. The 20-bit accumulator width is chosen so that repeated additions of the maximum 16-bit product (255 x 255 = 65,025) do not overflow across many accumulation cycles.

2. RTL Architecture

The design has a single combinational multiply feeding a single synchronous accumulate stage, matching the required interface exactly. The 8-bit operands A and B are multiplied combinationally to produce a 16-bit product. This product is zero-extended to 20 bits and added into the accumulator register Y on every rising clock edge, subject to the reset and enable control logic. Priority is reset first, then enable, then hold -- reset overrides enable so that asserting both simultaneously still clears the accumulator, matching the specification.

2.1 Module: mac8.v

```
module mac8 (
    input  wire        clk,
    input  wire        reset,
    input wire         enable,
    input  wire [7:0]  A,
    input wire [7:0]  B,
    output reg [19:0] Y
);

    wire [15:0] product;
    assign product = A * B;

    always @(posedge clk) begin
        if (reset)
            Y <= 20'd0;
        else if (enable)
            Y <= Y + {4'b0, product};
        else
            Y <= Y;
    end

endmodule
```

Design notes: the intermediate wire 'product' isolates the multiply as pure combinational logic with no clocking of its own -- this keeps the interface behaviourally correct in one clock cycle (Y updates on the same edge that enable is sampled). The zero-extension {4'b0, product} matches the 16-bit product width to the 20-bit accumulator width before addition. The explicit 'else Y <= Y' branch makes the hold behaviour unambiguous in simulation and synthesis alike.

3. Functional Verification

A self-checking testbench (mac8_tb.v) verifies the design against an independently written golden reference model rather than hand-computed expected values, so the same arithmetic mistake cannot silently appear in both the design and the check. The golden model is a second, simpler accumulator described directly from the specification. Every clock cycle, the testbench compares the DUT output Y against the golden model's output and tallies a running pass/fail count.

3.1 Test plan and coverage

Requirement	How it is covered in mac8_tb.v
a. Reset operation	Directed: reset alone; reset asserted simultaneously with enable=1 (reset must dominate)
b. Accumulation, enable=1	Directed: four consecutive accumulate cycles with varying A,B
c. Hold, enable=0	Directed: hold at start, hold mid-sequence, resume accumulation after hold
d. Various A,B combinations	Directed: seven cycles spanning small, large and mid-range operand pairs
e. Boundary cases	A=0, B=0, A=255, B=255 individually, plus repeated 255x255 to stress the 20-bit accumulator width
f. >=1000 random cases	1000-iteration loop using $random for A, B and enable, with periodic synchronous reset injection (every 137 cycles) to exercise reset during random traffic
Automatic pass/fail	Every cycle is compared against an independently written golden accumulator model; a running PASS/FAIL tally is printed and a final summary line reports overall result

3.2 Testbench: mac8_tb.v

```
`timescale 1ns/1ps

module mac8_tb;

    reg         clk, reset, enable;
    reg  [7:0]  A, B;
    wire [19:0] Y;

    integer pass_count, fail_count, check_count;
    reg [19:0] golden_Y;

    mac8 dut (
        .clk(clk), .reset(reset), .enable(enable),
        .A(A), .B(B), .Y(Y)
    );

    initial clk = 0;
    always #5 clk = ~clk;

    // Golden (reference) model -- independently written accumulator
    always @(posedge clk) begin
        if (reset)
            golden_Y <= 20'd0;
        else if (enable)
            golden_Y <= golden_Y + (A * B);
    end

    task check_result;
        begin
            check_count = check_count + 1;
```

```
            if (Y !== golden_Y) begin
                fail_count = fail_count + 1;
                $display("[FAIL] time=%0t A=%0d B=%0d DUT_Y=%0d EXPECTED_Y=%0d",
                         $time, A, B, Y, golden_Y);
            end else
                pass_count = pass_count + 1;
        end
    endtask

    task do_cycle(input r, input e, input [7:0] a, input [7:0] b);
        begin
            @(negedge clk);
            reset = r; enable = e; A = a; B = b;
            @(posedge clk);
            #1;
            check_result;
        end
    endtask

    integer i;
    reg [7:0] ra, rb;
    reg re, rr;

    initial begin
        pass_count = 0; fail_count = 0; check_count = 0;
        reset = 1; enable = 0; A = 0; B = 0; golden_Y = 0;

        // (a) reset
        do_cycle(1, 0, 0,   0);
        do_cycle(1, 1, 12,  34);   // reset must dominate even if enable=1

        // (b) accumulation
        do_cycle(0, 1, 3,   4);
        do_cycle(0, 1, 5,   6);
        do_cycle(0, 1, 10,  10);
        do_cycle(0, 1, 7,   8);

        // (c) hold
        do_cycle(0, 0, 99,  99);
        do_cycle(0, 0, 1,   1);
        do_cycle(0, 1, 2,   2);    // resume accumulation

        // (d) various A,B combinations
        do_cycle(1, 0, 0,   0);
        do_cycle(0, 1, 13,  17);
        do_cycle(0, 1, 200, 3);
        do_cycle(0, 1, 45,  67);
        do_cycle(0, 1, 128, 128);
        do_cycle(0, 0, 1,   1);    // hold in the middle
        do_cycle(0, 1, 99,  101);

        // (e) boundary cases
        do_cycle(1, 0, 0,   0);
        do_cycle(0, 1, 0,   0);
        do_cycle(0, 1, 0,   255);
        do_cycle(0, 1, 255, 0);
        do_cycle(0, 1, 255, 255);
        do_cycle(0, 1, 255, 255);
        do_cycle(0, 1, 255, 255);
```

```
        // (f) 1000+ randomized cases
        do_cycle(1, 0, 0, 0);
        for (i = 0; i < 1000; i = i + 1) begin
            ra = $random; rb = $random;
            re = $random % 2;
            rr = ((i % 137) == 0) ? 1'b1 : 1'b0;
            do_cycle(rr, re, ra, rb);
        end

        $display("TEST SUMMARY: %0d checks, %0d PASS, %0d FAIL", check_count, pass_count,
fail_count);
        if (fail_count == 0)
            $display("RESULT: ALL TESTS PASSED");
        else
            $display("RESULT: %0d TEST(S) FAILED", fail_count);
        $finish;
    end

endmodule
```

3.3 Simulation results

Compiled and simulated using Icarus Verilog (iverilog / vvp) on Ubuntu 24.04 (WSL):

```
$ iverilog -o sim.out rtl/mac8.v tb/mac8_tb.v
$ vvp sim.out
TEST SUMMARY: 1024 checks, 1024 PASS, 0 FAIL
RESULT: ALL TESTS PASSED
tb/mac8_tb.v:104: $finish called at 10246000 (1ps)
```

All 1024 checks (24 directed cases covering reset, accumulation, hold, varied operand combinations and boundary cases, plus 1000 randomized cases with periodic reset injection) passed with zero failures. This satisfies every sub-requirement of Part 1 (a) through (f), including automatic pass/fail reporting.

3.4 Waveform Evidence

The waveform below was captured in GTKWave from the VCD dump generated during simulation ($dumpfile/$dumpvars added to the testbench). It shows the boundary-case sequence where A=255, B=255 is applied for three consecutive enabled cycles -- the case most likely to expose an overflow or width-mismatch bug, since it repeatedly adds the maximum possible 16-bit product (255 x 255 = 65,025) into the accumulator.

```
navya@NavyaBetha:~/vlsi_project/mac_assignment$ iverilog -o sim.out rtl/mac8.v tb/mac8_tb.v
vvp sim.out
ls -la mac8_tb.vcd
VCD info: dumpfile mac8_tb.vcd opened for output.
TEST SUMMARY: 1024 checks, 1024 PASS, 0 FAIL
RESULT: ALL TESTS PASSED
tb/mac8_tb.v:108: $finish called at 10246000 (1ps)
-rw-r--r-- 1 navya navya 220969 Sep 23 04:19 mac8_tb.vcd
```

Figure 1. Terminal log: compilation and simulation, 1024/1024 checks passed, 0 failures.

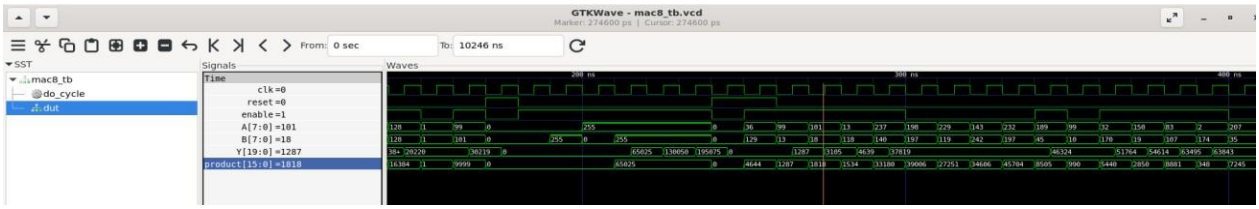


Figure 2. Full-run waveform overview showing clk, reset, enable, A, B, Y and product across the directed and boundary test sections.

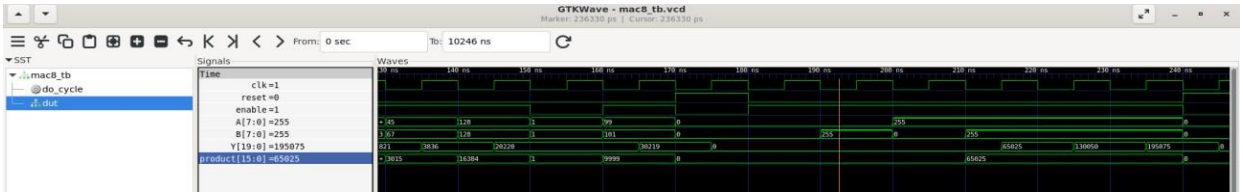


Figure 3. Zoomed view centred on the A=255, B=255 boundary-case block, marker placed at 236,330 ps.

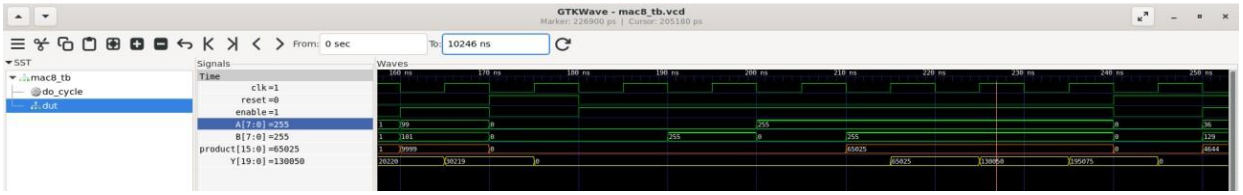


Figure 4. Final annotated capture (160-250 ns) isolating the three consecutive accumulate edges used for verification below.

Reading Figure 4 cycle by cycle confirms the recurrence Y(n+1) = Y(n) + A x B exactly:

Edge	A	B	product = A x B	Y before edge	Y after edge
1	255	255	65025	0 (Y(n))	65025 (Y(n+1))
2	255	255	65025	65025 (Y(n))	130050 (Y(n+1))
3	255	255	65025	130050 (Y(n))	195075 (Y(n+1))

Note on single-register semantics: Y(n) and Y(n+1) are not two separate signals in the RTL or the waveform -- they are the same physical register Y sampled one clock edge apart. The value visible on Y immediately before a rising edge is Y(n); the same register, immediately after that edge, holds Y(n+1). That new value then persists (held) until the next enabled edge, at which point it becomes Y(n) for the next accumulation. The waveform's continuous 0 -> 65025 -> 130050 -> 195075 progressions on the single Y trace is exactly this relationship in action, and matches the golden-model comparison already reported as 1024/1024 PASS in the simulation log.

4. Part 1 Summary

The baseline 8-bit MAC unit is implemented exactly to the specified interface and behaviour, and is functionally verified with a comprehensive, self-checking, randomized testbench. This RTL and testbench form the input to Part 2 (synthesis against the SKY130 standard-cell library) and Part 3 (architecture optimization), covered in the following sections.

Part 2: Baseline Synthesis and Timing Analysis

2.1 Baseline Architecture

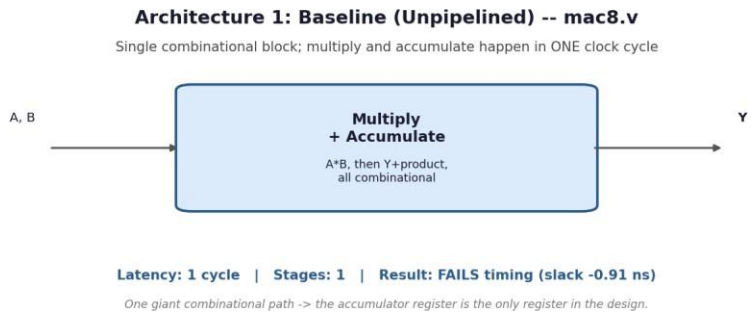


Figure 4a. Baseline (unpipelined) architecture: multiply and accumulate in one combinational block, one register.

Before looking at synthesis numbers, it helps to see the architecture being measured: the entire multiply-accumulate operation happens within a single clock period, with only the 20-bit accumulator register providing any state. This is the simplest possible correct implementation of the required interface, and the starting point every subsequent optimization is measured against.

2.2 Synthesis Methodology

The baseline (unpipelined) mac8 design was synthesized using Yosys 0.33 against the real SKY130 high-density standard-cell library (sky130_fd_sc_hd, typical corner, tt_025C_1v80), obtained directly from the SkyWater PDK GitHub mirror. The flow performs generic RTL synthesis, maps sequential elements to the library's D flip-flop cell, then runs ABC for technology-independent-to-technology-mapped gate selection.

2.2.1 Synthesis TCL script (synth_baseline.ys)

```
read_liberty -lib config/sky130_fd_sc_hd__tt_025C_1v80.lib
read_verilog rtl/mac8.v

synth -top mac8

dfflibmap -liberty config/sky130_fd_sc_hd__tt_025C_1v80.lib
abc -liberty config/sky130_fd_sc_hd__tt_025C_1v80.lib

clean
write_verilog reports/mac8_synth_baseline.v

stat -liberty config/sky130_fd_sc_hd__tt_025C_1v80.lib
```

Run with: yosys -s config/synth_baseline.ys > reports/synthesis_baseline.rpt 2>&1

2.3 Baseline Synthesis Results

```
navya@NavyaBetha:~/vlsi_project/mac_assignment$ nano config/synth_baseline.ys
navya@NavyaBetha:~/vlsi_project/mac_assignment$ yosys -s config/synth_baseline.ys > reports/synthesis_baseline.rpt 2>&1
tail -60 reports/synthesis_baseline.rpt
6.1. Executing BMUXMAP pass.

6.2. Executing DEMUXMAP pass.
Dumping module `\mac8'.

7. Printing statistics.

=== mac8 ===

   Number of wires:                383
   Number of wire bits:            416
   Number of public wires:           6
   Number of public wire bits:      39
   Number of memories:               0
   Number of memory bits:            0
   Number of processes:              0
   Number of cells:                397
```

Figure 5. Baseline synthesis -- top of the cell statistics report (397 cells total).



Figure 6. Baseline synthesis -- cell-type breakdown tail and final chip area (3169.29 µm²).

The baseline design synthesizes to 397 standard cells (20 sky130_fd_sc_hd__dfxtp_1 flip-flops for the 20-bit accumulator, 377 combinational cells) occupying 3169.29 µm². Notably, the cell list includes 34 maj3 (majority-of-3) and 27+8 xor2/xor3 instances -- these are literally the carry and sum logic of full-adder cells, confirming that ABC implemented the multiplier's reduction array and the accumulator's adder using compressed full-adder-equivalent gates rather than discrete AND/OR/XOR primitives.

2.4 Timing Constraints and Analysis Methodology

Timing was analyzed with OpenSTA 2.4.0, run inside the OpenLane Docker container, using the synthesized gate-level netlist and the real SKY130 liberty timing arcs (no wireload/parasitics -- this is synthesis-stage, pre-placement timing; post-route timing is obtained in Part 4).

2.4.1 Timing constraints (constraints/constraints.sdc)

```
create_clock -name clk -period 4.0 [get_ports clk]
set_input_delay  -clock clk 0.8 [get_ports {reset enable A B}]
set_output_delay -clock clk 0.8 [get_ports Y]
set_clock_uncertainty 0.15 [get_clocks clk]
```

This sets the target clock period to 4.0 ns (250 MHz), a 0.8 ns input/output delay budget on all data ports, and 0.15 ns of clock uncertainty margin for jitter/skew.

2.4.2 OpenSTA commands

```
read_liberty config/sky130_fd_sc_hd__tt_025C_1v80.lib
read_verilog reports/mac8_synth_baseline.v
link_design mac8
read_sdc constraints/constraints.sdc
report_checks -path_delay max
```

2.5 Baseline Timing Results

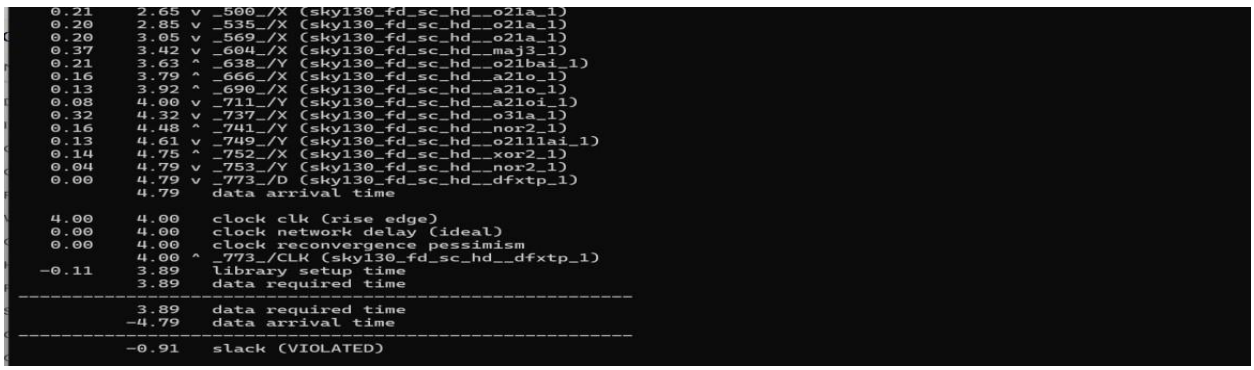


Figure 7. Baseline OpenSTA report –cirtical path-- final timing summary: slack = −0.91 ns (VIOLATED).

Part 2(a) -- Does the baseline meet 250 MHz?

No. OpenSTA reports a data arrival time of 4.79 ns against a data required time of 3.89 ns, giving a slack of −0.91 ns (VIOLATED). The design as synthesized cannot run reliably at 250 MHz (4 ns period); its maximum safe frequency is approximately 1/4.79ns + setup margin ≈ 205–210 MHz.

Part 2(b) -- Which block dominates the critical path?

The critical path is an 18-cell unbroken combinational chain (xnor3 → maj3 → xor3 → or4b → maj3 → o21a×3 → maj3 → o21bai → a21o×2 → a21oi → o31a → nor2 → o2111ai → xor2 → nor2), running from the accumulator register, through the multiplier's partial-product reduction array (identifiable by the maj3/xor3 full-adder-equivalent cells), directly into the 20-bit accumulator's own carry-propagate adder, with no register boundary between them. The multiply-then-accumulate combinational cone as a whole is the dominant contributor -- specifically, the fact that both operations occur in a single unbroken chain within one clock period.

Part 2(c) -- Architecture modification

Since the baseline violates the 4 ns requirement, the architecture was pipelined. Section 3 documents five design iterations explored (2, 3, 4 and 5 pipeline stages), the specific timing failure diagnosed at each step using real OpenSTA data, and the final 5-stage carry-save pipelined architecture that meets timing with +0.10 ns of positive slack.

Part 3: Architecture Optimization -- Pipelining

3.1 Pipeline Architecture Background

Several distinct pipelining strategies exist for arithmetic datapaths, and choosing the wrong one for an accumulate structure can silently break correctness or, as this project demonstrates, fail to improve (or even worsen) timing:

- Structural / linear (feed-forward) pipelining: registers are inserted between stages of a chain that only flows forward (e.g. splitting a multiplier's array into two register-separated halves). Safe by default, since no stage depends on a result computed later in the pipeline.
- Loop-carried (feedback) pipelining: applies when a stage's output feeds back as an input to an earlier or the same stage on a later cycle -- exactly the case for an accumulator, where Y(n+1) depends on Y(n). Naively inserting registers into a feedback loop changes which value is read back (a read-after-write hazard) and produces silently wrong results if not handled carefully.
- Carry-save (redundant number system) pipelining: the technique used to pipeline an accumulation loop correctly and with an O(1), width-independent per-stage delay. The accumulator is kept in a redundant sum/carry (S, C) form rather than resolved binary; adding a new term each cycle is a layer of independent per-bit full-adder logic (a 3:2 compressor) with no carry propagation within the cycle, so it can execute every cycle regardless of accumulator width. The true binary value is resolved (S + C) only when needed at the output, in a stage that feeds forward only and therefore imposes no correctness constraint on the loop.
- Balanced/retimed pipelining: further subdividing an already feed-forward stage (such as a final resolve adder) into smaller chunks with intermediate registers, purely to shorten the worst combinational delay per stage -- used here for Stages 3–5.

This project used a combination of linear pipelining (Stage 1, multiply) and carry-save pipelining (Stage 2, accumulate) to correctly break the accumulator's feedback loop, followed by balanced pipelining (Stages 3–5) to shorten the remaining feed-forward resolve adder. Carry-save was chosen specifically because it is the standard, textbook-correct way to pipeline an accumulation loop without introducing a read-after-write hazard -- a naive split of the ripple-carry adder itself across pipeline stages would have required reading the not-yet-written previous result on the very next cycle, which cannot be done safely with simple registers.

3.2 Design Space Explored

Five architectures were implemented, functionally verified against independently-written golden models, synthesized against the real SKY130 library, and timed with OpenSTA at the 4 ns / 250 MHz target. All results below are taken directly from tool output, not estimated.

Design	Stages	Latency (cyc)	Cells	Area (µm²)	FFs	Slack (ns)	Status
Baseline	1	1	397	3169.29	20	−0.91	VIOLATED
2-stage (naive split)	2	2	433	3328.19	37	−2.74	VIOLATED (worse)
3-stage (carry-save)	3	3	593	4764.57	76	−2.91	VIOLATED
4-stage (resolve split 10+10)	4	4	637	5707.97	107	−0.21	VIOLATED
5-stage (resolve split 5+5+10)	5	5	690	6389.88	133	+0.10	MET

3.2.0 Visual Overview: Area and Timing Across All Iterations

Before the detailed per-iteration discussion below, the two charts here summarise the whole design-space exploration at a glance: how area grew (and then recovered) across the seven RTL variants, and how timing slack moved from violated to met and back to violated (post-route) before finally closing for good.

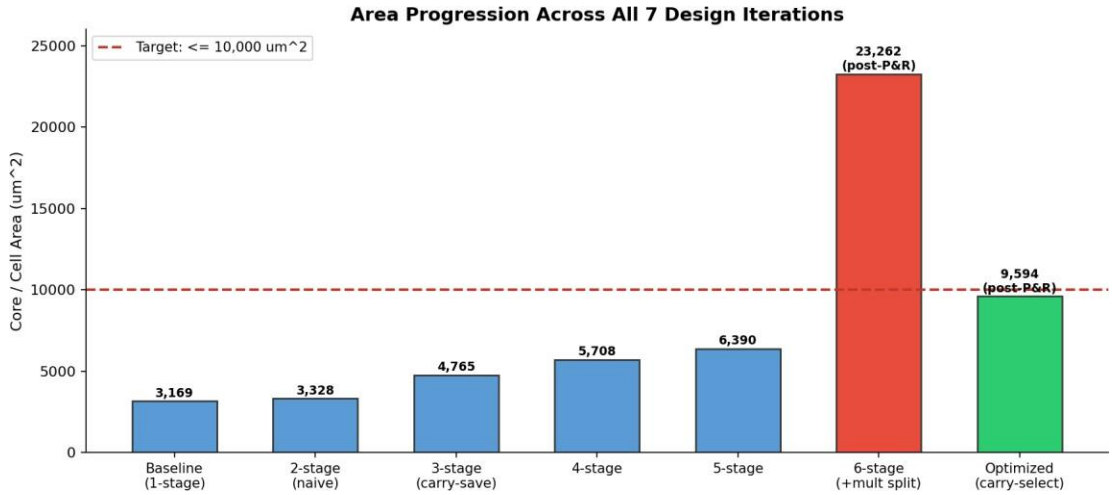


Figure 8. Area progression across all seven design iterations, from baseline through the final optimized carry-select design.

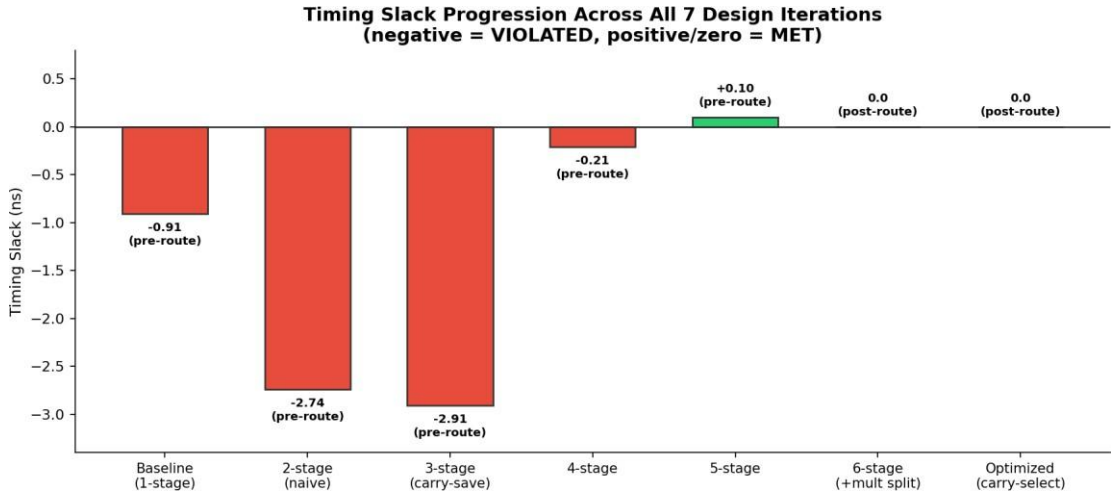


Figure 9. Timing slack progression across all seven design iterations -- note Iteration 1 (2-stage) going MORE negative than the baseline, direct evidence that pipelining in the wrong place worsens timing.

3.2.1 Iteration 1: 2-stage naive pipeline (multiply | accumulate)

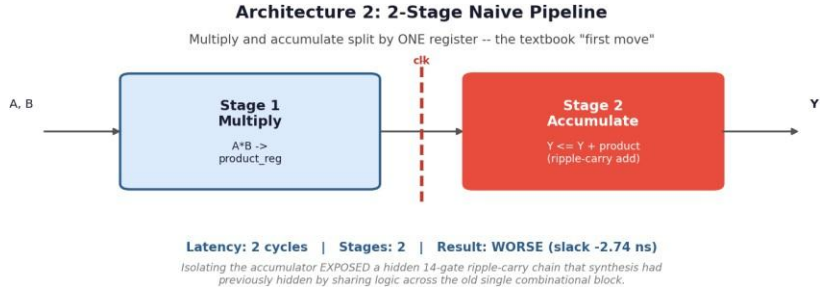


Figure 9a. 2-stage naive pipeline architecture: multiply and accumulate separated by one register.

The first attempt registered the multiplier's output, isolating multiply and accumulate into separate pipeline stages -- the textbook first move for a MAC unit.

```
module mac8_pipelined (
    input  wire         clk,
    input  wire         reset,
    input  wire         enable,
    input  wire [7:0]  A,
    input  wire [7:0]  B,
    output reg  [19:0] Y
);
    // ---- Stage 1: multiply ----
    reg  [15:0] product_reg;
    reg         enable_stage2;
    wire [15:0] product = A * B;

    always @(posedge clk) begin
        if (reset) begin
            product_reg    <= 16'd0;
            enable_stage2 <= 1'b0;
        end else begin
            product_reg    <= product;
            enable_stage2 <= enable;
        end
```

```
    end

    // ---- Stage 2: accumulate (plain ripple-carry add) ----
    always @(posedge clk) begin
        if (reset)
            Y <= 20'd0;
        else if (enable_stage2)
            Y <= Y + {4'b0, product_reg};
        else
            Y <= Y;
    end
endmodule
```

Synthesis script (config/synth_pipelined.ys) follows the same template as Section 2.1.1 with rtl/mac8_pipelined.v and -top mac8_pipelined.

```
navya@NavyaBetha:~/vlsi_project/mac_assignment$ nano config/synth_pipelined.ys
navya@NavyaBetha:~/vlsi_project/mac_assignment$ yosys -s config/synth_pipelined.ys > reports/synthesis_pipelined.rpt 2>&
1
tail -60 reports/synthesis_pipelined.rpt
Removing temp directory.
Removed 0 unused cells and 560 unused wires.

6. Executing Verilog backend.

6.1. Executing BMUXMAP pass.

6.2. Executing DEMUXMAP pass.
Dumping module '\mac8_pipelined'.

7. Printing statistics.

=== mac8_pipelined ===

   Number of wires:                404
   Number of wire bits:            452
   Number of public wires:           8
   Number of public wire bits:      56
   Number of memories:               0
   Number of memory bits:            0
   Number of processes:              0
   Number of cells:                433
     sky130_fd_sc_hd__a211oi_1       2
```

Figure 9. 2-stage pipeline -- synthesis begins, 433 total cells.

```
     sky130_fd_sc_hd__and3b_1        1
     sky130_fd_sc_hd__and4_1         6
     sky130_fd_sc_hd__clkinv_1       5
     sky130_fd_sc_hd__dfxtp_1       37
     sky130_fd_sc_hd__lpflow_inputiso1p_1      3
     sky130_fd_sc_hd__lpflow_isobufsrc_1     12
     sky130_fd_sc_hd__maj3_1         25
     sky130_fd_sc_hd__nand2_1        41
     sky130_fd_sc_hd__nand2b_1        9
     sky130_fd_sc_hd__nand3_1         5
     sky130_fd_sc_hd__nand4_1        15
     sky130_fd_sc_hd__nor2_1         47
     sky130_fd_sc_hd__nor2b_1         5
     sky130_fd_sc_hd__nor3_1         29
     sky130_fd_sc_hd__o21a_1         10
     sky130_fd_sc_hd__o21ai_0        20
     sky130_fd_sc_hd__o21bai_1        1
     sky130_fd_sc_hd__or3_1           2
     sky130_fd_sc_hd__xnor2_1        37
     sky130_fd_sc_hd__xnor3_1         2
     sky130_fd_sc_hd__xor2_1         40
     sky130_fd_sc_hd__xor3_1          4

   Chip area for module '\mac8_pipelined': 3328.192000

Warnings: 26 unique messages, 234 total
End of script. Logfile hash: 10eb718ef8, CPU: user 0.51s system 0.02s, MEM: 39.68 MB peak
Yosys 0.33 (git sha1 2584903a060)
Time spent: 33% 2x abc (0 sec), 14% 2x stat (0 sec), ...
navya@NavyaBetha:~/vlsi_project/mac_assignment$
```

Figure 10. 2-stage pipeline -- final cell breakdown and chip area (3328.19 µm², 37 flip-flops).

```
navya@NavyaBetha:~/vlsi_project/mac_assignment$ nano tb/mac8_pipelined_tb.v
navya@NavyaBetha:~/vlsi_project/mac_assignment$ iverilog -o sim_pipe.out rtl/mac8_pipelined.v tb/mac8_pipelined_tb.v
navya@NavyaBetha:~/vlsi_project/mac_assignment$ vvp sim_pipe.out
TEST SUMMARY: 1008 checks, 1008 PASS, 0 FAIL
RESULT: ALL TESTS PASSED
tb/mac8_pipelined_tb.v:91: $finish called at 10086000 (1ps)
navya@NavyaBetha:~/vlsi_project/mac_assignment$
```

Figure 11. 2-stage pipeline functional verification: 1008/1008 checks passed.

OpenSTA result (no screenshot captured for this iteration; full text output reproduced below for completeness):

```
Startpoint: _757_ (rising edge-triggered flip-flop clocked by clk)
Endpoint: _773_ (rising edge-triggered flip-flop clocked by clk)
Path Group: clk / Path Type: max
   0.00    0.00 ^ _757_/CLK (sky130_fd_sc_hd__dfxtp_1)
   0.34    0.34 v _757_/Q
   ...  fourteen sky130_fd_sc_hd__maj3_1 cells chained back-to-back
        (plain ripple-carry accumulator, now isolated as its own stage) ...
   0.00    6.47 v _773_/D (sky130_fd_sc_hd__dfxtp_1)
           6.47   data arrival time
           3.73   data required time
-----------------------------------------------------------
          -2.74   slack (VIOLATED)
```

Unexpected result: slack got WORSE (−2.74 ns vs. baseline's −0.91 ns), even though area grew only modestly. The critical path is fourteen sky130_fd_sc_hd__maj3_1 cells chained back-to-back -- a textbook ripple-carry adder with no logic sharing. In the baseline, ABC's optimizer had one large combined multiply+accumulate netlist to restructure and found shorter alternate paths through it; once isolated into its own stage, the accumulator's plain ripple-carry structure was exposed with nothing to hide behind. This demonstrates that pipelining does not automatically improve timing -- it only helps if the resulting stage boundaries don't expose an already-weak sub-block (see Section 3.4, Question 3b).

3.2.2 Iteration 2: 3-stage carry-save pipeline

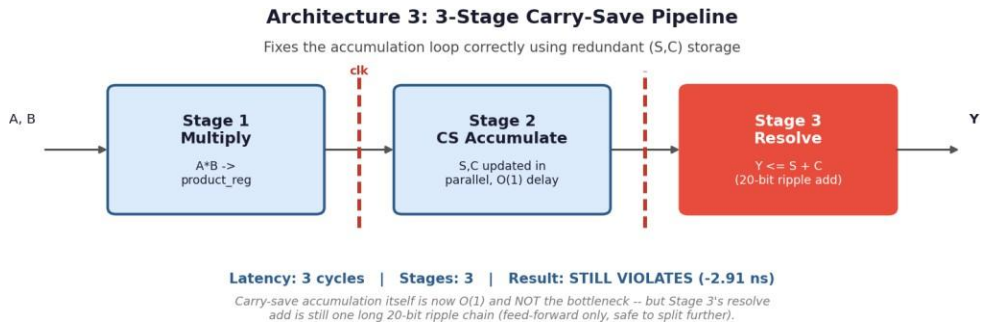


Figure 12a. 3-stage carry-save pipeline architecture: correctly fixes the loop-carried accumulation hazard.

To fix the exposed ripple-carry adder correctly (without introducing a read-after-write hazard on the accumulation loop), the accumulator was redesigned in carry-save form.

```
module mac8_pipelined3 (
    input  wire         clk,
    input  wire         reset,
    input  wire         enable,
    input  wire [7:0]  A,
    input  wire [7:0]  B,
    output reg  [19:0] Y
);
    // ---- Stage 1: multiply ----
    reg  [15:0] product_reg;
    reg         en1;
    wire [15:0] product = A * B;

    always @(posedge clk) begin
        if (reset) begin
            product_reg <= 16'd0;
```

```
            en1         <= 1'b0;
        end else begin
            product_reg <= product;
            en1         <= enable;
        end
    end

    // ---- Stage 2: carry-save accumulate -- O(1) delay, NO ripple ----
    reg  [19:0] S, C;    // true accumulator value = S + C (redundant form)
    wire [19:0] X = {4'b0, product_reg};
    wire [19:0] S_next;
    wire [19:0] C_next;

    genvar i;
    generate
        for (i = 0; i < 20; i = i + 1) begin : csa_gen
            assign S_next[i] = S[i] ^ C[i] ^ X[i];
            if (i == 0)
                assign C_next[i] = 1'b0;
            else
                assign C_next[i] = (S[i-1] & C[i-1]) | (S[i-1] & X[i-1]) | (C[i-1] & X[i-1]);
        end
    endgenerate

    always @(posedge clk) begin
        if (reset) begin
            S <= 20'd0;
            C <= 20'd0;
        end else if (en1) begin
            S <= S_next;
            C <= C_next;
        end
    end

    // ---- Stage 3: resolve true binary value -- feed-forward only ----
    always @(posedge clk) begin
        if (reset)
            Y <= 20'd0;
        else
            Y <= S + C;   // does NOT feed back into the accumulation loop
    end
endmodule
```

```
read_liberty -lib config/sky130_fd_sc_hd__tt_025C_1v80.lib
read_verilog rtl/mac8_pipelined3.v

synth -top mac8_pipelined3

dfflibmap -liberty config/sky130_fd_sc_hd__tt_025C_1v80.lib
abc -liberty config/sky130_fd_sc_hd__tt_025C_1v80.lib

clean
write_verilog reports/mac8_synth_pipelined3.v

stat -liberty config/sky130_fd_sc_hd__tt_025C_1v80.lib
```

```
navya@NavyaBetha:~/vlsi_project/mac_assignment$ nano tb/mac8_pipelined3_tb.v
navya@NavyaBetha:~/vlsi_project/mac_assignment$ iverilog -o sim_pipe3.out rtl/mac8_pipelined3.v tb/mac8_pipelined3_tb.v
vvp sim_pipe3.out
TEST SUMMARY: 1009 checks, 1009 PASS, 0 FAIL
RESULT: ALL TESTS PASSED
tb/mac8_pipelined3_tb.v:116: $finish called at 10096000 (1ps)
navya@NavyaBetha:~/vlsi_project/mac_assignment$ nano config/synth_pipelined3.ys
```

Figure 12. 3-stage carry-save pipeline functional verification: 1009/1009 checks passed.

```
=== mac8_pipelined3 ===

   Number of wires:                529
   Number of wire bits:            653
   Number of public wires:          12
   Number of public wire bits:     136
   Number of memories:               0
   Number of memory bits:            0
   Number of processes:              0
   Number of cells:                593
```

Figure 13. 3-stage pipeline -- synthesis statistics header, 593 total cells.

```
  sky130_fd_sc_hd__nand3_1        8
  sky130_fd_sc_hd__nand4_1       14
  sky130_fd_sc_hd__nor2_1        92
  sky130_fd_sc_hd__nor2b_1        4
  sky130_fd_sc_hd__nor3_1        27
  sky130_fd_sc_hd__nor3b_1        1
  sky130_fd_sc_hd__o21a_1        13
  sky130_fd_sc_hd__o21ai_0       26
  sky130_fd_sc_hd__o21ba_1        1
  sky130_fd_sc_hd__o21bai_1       2
  sky130_fd_sc_hd__or3_1          1
  sky130_fd_sc_hd__or3b_1         1
  sky130_fd_sc_hd__xnor2_1       52
  sky130_fd_sc_hd__xnor3_1        1
  sky130_fd_sc_hd__xor2_1        52
  sky130_fd_sc_hd__xor3_1         1

Chip area for module '\mac8_pipelined3': 4764.569600
```

Figure 14. 3-stage pipeline -- final cell breakdown and chip area (4764.57 µm², 76 flip-flops).

```
   0.39    2.99 v _0707_/X (sky130_fd_sc_hd__maj3_1)
   0.39    3.38 v _0710_/X (sky130_fd_sc_hd__maj3_1)
   0.39    3.77 v _0713_/X (sky130_fd_sc_hd__maj3_1)
   0.39    4.16 v _0716_/X (sky130_fd_sc_hd__maj3_1)
   0.39    4.56 v _0719_/X (sky130_fd_sc_hd__maj3_1)
   0.39    4.95 v _0722_/X (sky130_fd_sc_hd__maj3_1)
   0.39    5.34 v _0725_/X (sky130_fd_sc_hd__maj3_1)
   0.39    5.73 v _0728_/X (sky130_fd_sc_hd__maj3_1)
   0.39    6.12 v _0731_/X (sky130_fd_sc_hd__maj3_1)
   0.09    6.21 ^ _0732_/Y (sky130_fd_sc_hd__nand2_1)
   0.08    6.29 v _0735_/Y (sky130_fd_sc_hd__a21oi_1)
   0.34    6.63 v _0739_/X (sky130_fd_sc_hd__or3b_1)
   0.13    6.76 ^ _0740_/Y (sky130_fd_sc_hd__a21oi_1)
   0.00    6.76 ^ _1093_/D (sky130_fd_sc_hd__dfxtp_1)
           6.76   data arrival time

   4.00    4.00   clock clk (rise edge)
   0.00    4.00   clock network delay (ideal)
  -0.10    3.90   clock uncertainty
   0.00    3.90   clock reconvergence pessimism
           3.90 ^ _1093_/CLK (sky130_fd_sc_hd__dfxtp_1)
  -0.06    3.84   library setup time
           3.84   data required time
-----------------------------------------------------------
           3.84   data required time
          -6.76   data arrival time
-----------------------------------------------------------
          -2.91   slack (VIOLATED)
```

Figure 15. 3-stage pipeline OpenSTA result: slack = −2.91 ns (VIOLATED).

The carry-save accumulation loop itself is now correctly O(1) delay and is confirmed NOT to be the critical path. However, slack is still negative (−2.91 ns) and the critical path is again fourteen maj3_1 cells in a row -- this time from Stage 3's Y <= S + C resolve step, which is still a plain 20-bit ripple-carry adder. Since this stage only drives the output port (never feeds back into S/C), it is safe to split further with no hazard risk.

3.2.3 Iteration 3: 4-stage pipeline (resolve adder split 10+10)

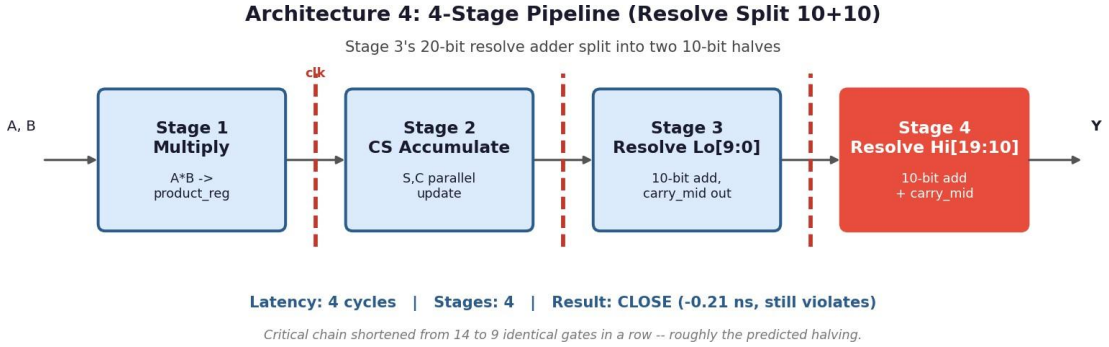


Figure 16a. 4-stage pipeline architecture: the 20-bit resolve adder split into two 10-bit halves.

Stage 3's 20-bit resolve adder was split into two 10-bit halves with a registered carry passed between them.

```
module mac8_pipelined4 (
    input  wire        clk,
    input  wire        reset,
    input  wire        enable,
    input  wire [7:0]  A,
    input  wire [7:0]  B,
    output reg  [19:0] Y
);
    // Stage 1 (multiply) and Stage 2 (carry-save accumulate) are
    // IDENTICAL to mac8_pipelined3 -- see Section 3.3.2. Omitted here
    // for brevity; full listing in the project source tree.

    // ---- Stage 3: resolve LOW 10 bits, register carry into high half ----
    reg  [9:0]  sum_lo_reg;
    reg         carry_mid;
    reg  [9:0]  S_hi_reg, C_hi_reg;

    always @(posedge clk) begin
        if (reset) begin
            sum_lo_reg <= 10'd0;
            carry_mid  <= 1'b0;
            S_hi_reg   <= 10'd0;
            C_hi_reg   <= 10'd0;
        end else begin
            {carry_mid, sum_lo_reg} <= S[9:0] + C[9:0];
            S_hi_reg <= S[19:10];
            C_hi_reg <= C[19:10];
        end
    end

    // ---- Stage 4: resolve HIGH 10 bits using the registered carry-in ----
    always @(posedge clk) begin
        if (reset)
            Y <= 20'd0;
        else
            Y <= {(S_hi_reg + C_hi_reg + carry_mid), sum_lo_reg};
    end
```

```
endmodule
```

```
read_liberty -lib config/sky130_fd_sc_hd__tt_025C_1v80.lib
read_verilog rtl/mac8_pipelined4.v

synth -top mac8_pipelined4

dfflibmap -liberty config/sky130_fd_sc_hd__tt_025C_1v80.lib
abc -liberty config/sky130_fd_sc_hd__tt_025C_1v80.lib

clean
write_verilog reports/mac8_synth_pipelined4.v

stat -liberty config/sky130_fd_sc_hd__tt_025C_1v80.lib
```

```
navya@NavyaBetha:~/vlsi_project/mac_assignment$ nano tb/mac8_pipelined4_tb.v
navya@NavyaBetha:~/vlsi_project/mac_assignment$ iverilog -o sim_pipe4.out rtl/mac8_pipelined4.v tb/mac8_pipelined4_tb.v
vvp sim_pipe4.out
TEST SUMMARY: 1009 checks, 1009 PASS, 0 FAIL
RESULT: ALL TESTS PASSED
tb/mac8_pipelined4_tb.v:132: $finish called at 10096000 (1ps)
navya@NavyaBetha:~/vlsi_project/mac_assignment$ nano config/synth_pipelined4.ys
navya@NavyaBetha:~/vlsi_project/mac_assignment$ yosys -s config/synth_pipelined4.ys > reports/synthesis_pipelined4.rpt 2>&1
tail -60 reports/synthesis_pipelined4.rpt
```

Figure 16. 4-stage pipeline functional verification: 1009/1009 checks passed.

```
navya@NavyaBetha: ~/vlsi_pr

6.2. Executing DEMUXMAP pass.
Dumping module `\mac8_pipelined4'.

7. Printing statistics.

=== mac8_pipelined4 ===

   Number of wires:                546
   Number of wire bits:            697
   Number of public wires:          16
   Number of public wire bits:     167
   Number of memories:               0
   Number of memory bits:            0
   Number of processes:              0
   Number of cells:                637
```

Figure 17. 4-stage pipeline -- synthesis statistics, 637 total cells.

```
navya@NavyaBetha: ~/vlsi_pr

    sky130_fd_sc_hd__nor2_1        77
    sky130_fd_sc_hd__nor2b_1        3
    sky130_fd_sc_hd__nor3_1        18
    sky130_fd_sc_hd__nor4b_1        1
    sky130_fd_sc_hd__o211a_1        7
    sky130_fd_sc_hd__o21a_1         7
    sky130_fd_sc_hd__o21ai_0       34
    sky130_fd_sc_hd__o21bai_1       2
    sky130_fd_sc_hd__xnor2_1       38
    sky130_fd_sc_hd__xnor3_1        5
    sky130_fd_sc_hd__xor2_1        45
    sky130_fd_sc_hd__xor3_1        14

   Chip area for module '\mac8_pipelined4': 5707.974400

Warnings: 26 unique messages, 234 total
End of script. Logfile hash: 24173725f1, CPU: user 0.83s system 0.04s, MEM: 40.17 MB peak
Yosys 0.33 (git sha1 2584903a060)
Time spent: 32% 2x abc (0 sec), 13% 2x stat (0 sec), ...
```

Figure 18. 4-stage pipeline -- final cell breakdown and chip area (5707.97 µm², 107 flip-flops).

```
   0.00    0.00   clock network delay (ideal)
   0.00    0.00 ^ _1110_/CLK (sky130_fd_sc_hd__dfxtp_1)
   0.33    0.33 v _1110_/Q (sky130_fd_sc_hd__dfxtp_1)
   0.38    0.71 v _0632_/X (sky130_fd_sc_hd__maj3_1)
   0.39    1.10 v _0636_/X (sky130_fd_sc_hd__maj3_1)
   0.39    1.49 v _0640_/X (sky130_fd_sc_hd__maj3_1)
   0.39    1.88 v _0644_/X (sky130_fd_sc_hd__maj3_1)
   0.39    2.28 v _0648_/X (sky130_fd_sc_hd__maj3_1)
   0.39    2.67 v _0652_/X (sky130_fd_sc_hd__maj3_1)
   0.39    3.06 v _0656_/X (sky130_fd_sc_hd__maj3_1)
   0.39    3.45 v _0660_/X (sky130_fd_sc_hd__maj3_1)
   0.37    3.82 v _0664_/X (sky130_fd_sc_hd__maj3_1)
   0.15    3.97 ^ _0666_/Y (sky130_fd_sc_hd__xnor2_1)
   0.04    4.01 v _0667_/Y (sky130_fd_sc_hd__nor2_1)
   0.00    4.01 v _1099_/D (sky130_fd_sc_hd__dfxtp_1)
           4.01   data arrival time

   4.00    4.00   clock clk (rise edge)
   0.00    4.00   clock network delay (ideal)
  -0.10    3.90   clock uncertainty
   0.00    3.90   clock reconvergence pessimism
           3.90 ^ _1099_/CLK (sky130_fd_sc_hd__dfxtp_1)
  -0.09    3.81   library setup time
           3.81   data required time
-----------------------------------------------------------
           3.81   data required time
          -4.01   data arrival time
-----------------------------------------------------------
          -0.21   slack (VIOLATED)
```

Figure 19. 4-stage pipeline OpenSTA result: slack = −0.21 ns (VIOLATED, but close).

The critical path chain shortened from 14 to 9 maj3_1 cells -- roughly the halving predicted by splitting a 20-bit ripple chain into two 10-bit chains. Slack improved dramatically to −0.21 ns, about 5% short of closing.

3.2.4 Iteration 4 (final): 5-stage pipeline (resolve adder split 5+5+10)

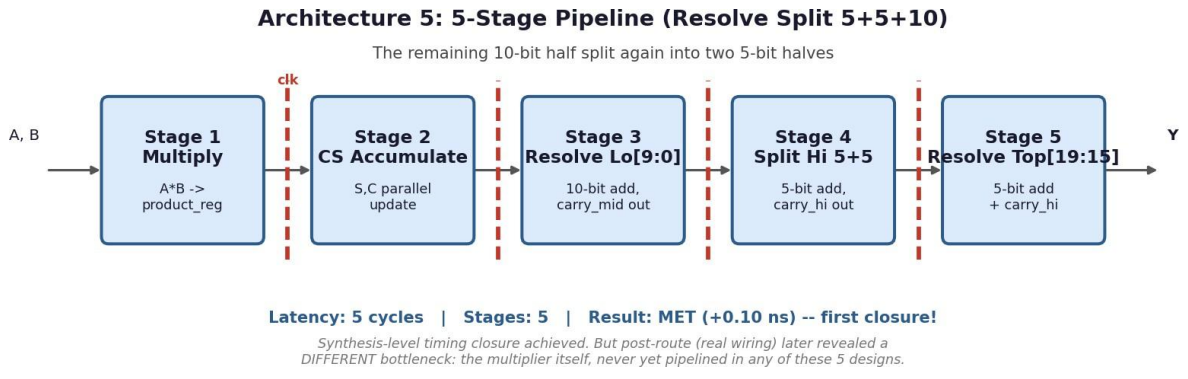


Figure 19a. 5-stage pipeline architecture: the remaining 10-bit half split again into two 5-bit halves.

The remaining 10-bit high-half resolve add was split again into two 5-bit halves, fully isolating the last long ripple chain.

```
module mac8_pipelined5 (
    input  wire         clk,
    input  wire         reset,
    input  wire         enable,
    input  wire [7:0]  A,
    input  wire [7:0]  B,
    output reg  [19:0] Y
);

/* Stage 1 (multiply) and Stage 2 (carry-save accumulate) are IDENTICAL to mac8_pipelined3 --
see Section 3.3.2.  ---Stage 3: resolve LOW 10 bits, pass HIGH 10 bits forward-----*/

    reg  [9:0] sum_lo_reg;
    reg        carry_mid;
    reg  [9:0] S_hi_reg, C_hi_reg;

    always @(posedge clk) begin
        if (reset) begin
            sum_lo_reg <= 10'd0;
            carry_mid  <= 1'b0;
            S_hi_reg   <= 10'd0;
            C_hi_reg   <= 10'd0;
        end else begin
            {carry_mid, sum_lo_reg} <= S[9:0] + C[9:0];
            S_hi_reg <= S[19:10];
            C_hi_reg <= C[19:10];
        end
    end

    // ---- Stage 4: split the HIGH 10 bits into two 5-bit halves ----

    reg  [4:0] sum_mid_reg;
    reg        carry_hi;
    reg  [4:0] S_hi2_reg, C_hi2_reg;
    reg  [9:0] sum_lo_reg2;    // pass-through, keeps alignment with Y

    always @(posedge clk) begin
        if (reset) begin
            sum_mid_reg  <=  5'd0;
            carry_hi    <=   1'b0;
            S_hi2_reg   <=  5'd0;
            C_hi2_reg   <=  5'd0;
            sum_lo_reg2 <= 10'd0;
        end else begin
            {carry_hi, sum_mid_reg} <= S_hi_reg[4:0] + C_hi_reg[4:0] + carry_mid;
            S_hi2_reg   <= S_hi_reg[9:5];
            C_hi2_reg   <= C_hi_reg[9:5];
            sum_lo_reg2 <= sum_lo_reg;
        end
    end

    // ---- Stage 5: resolve TOP 5 bits, assemble final Y ----

    always @(posedge clk) begin
        if (reset)
            Y <= 20'd0;
        else
            Y <= {(S_hi2_reg + C_hi2_reg + carry_hi), sum_mid_reg, sum_lo_reg2};
    end
endmodule
```

```
read_liberty -lib config/sky130_fd_sc_hd__tt_025C_1v80.lib
read_verilog rtl/mac8_pipelined5.v

synth -top mac8_pipelined5

dfflibmap -liberty config/sky130_fd_sc_hd__tt_025C_1v80.lib
abc -liberty config/sky130_fd_sc_hd__tt_025C_1v80.lib

clean
write_verilog reports/mac8_synth_pipelined5.v

stat -liberty config/sky130_fd_sc_hd__tt_025C_1v80.lib
```

```
navya@NavyaBetha:~/vlsi_project/mac_assignment$ nano tb/mac8_pipelined5_tb.v
navya@NavyaBetha:~/vlsi_project/mac_assignment$ iverilog -o sim_pipe5.out rtl/mac8_pipelined5.v tb/mac8_pipelined5_tb.v
vvp sim_pipe5.out
TEST SUMMARY: 1009 checks, 1009 PASS, 0 FAIL
RESULT: ALL TESTS PASSED
tb/mac8_pipelined5_tb.v:155: $finish called at 10096000 (1ps)
navya@NavyaBetha:~/vlsi_project/mac_assignment$ nano config/synth_pipelined5.ys
navya@NavyaBetha:~/vlsi_project/mac_assignment$ yosys -s config/synth_pipelined5.ys > reports/synthesis_pipelined5.rpt 2>&1
tail -60 reports/synthesis_pipelined5.rpt

6. Executing Verilog backend.

6.1. Executing BMUXMAP pass.

6.2. Executing DEMUXMAP pass.
Dumping module `\mac8_pipelined5'.

7. Printing statistics.

=== mac8_pipelined5 ===

   Number of wires:                578
   Number of wire bits:            750
   Number of public wires:          21
   Number of public wire bits:     193
   Number of memories:               0
   Number of memory bits:            0
   Number of processes:              0
   Number of cells:                690
     sky130_fd_sc_hd__a211oi_1      13
     sky130_fd_sc_hd__a21boi_0       1
```

Figure 20. 5-stage pipeline -- functional verification (1009/1009 pass) and synthesis statistics header, 690 total cells.

```
   sky130_fd_sc_hd__a21o_1         7
   sky130_fd_sc_hd__a21oi_1       35
   sky130_fd_sc_hd__a22o_1         4
   sky130_fd_sc_hd__a22oi_1        7
   sky130_fd_sc_hd__a311oi_1       6
   sky130_fd_sc_hd__a31o_1         1
   sky130_fd_sc_hd__a31oi_1        1
   sky130_fd_sc_hd__and2_0         8
   sky130_fd_sc_hd__and3_1        12
   sky130_fd_sc_hd__and4_1         3
   sky130_fd_sc_hd__clkinv_1       3
   sky130_fd_sc_hd__dfxtp_1      133
   sky130_fd_sc_hd__lpflow_inputisolp_1      3
   sky130_fd_sc_hd__lpflow_isobufsrc_1      64
   sky130_fd_sc_hd__maj3_1         32
   sky130_fd_sc_hd__nand2_1        79
   sky130_fd_sc_hd__nand2b_1       12
   sky130_fd_sc_hd__nand3_1         8
   sky130_fd_sc_hd__nand4_1         9
   sky130_fd_sc_hd__nor2_1         77
   sky130_fd_sc_hd__nor2b_1         5
   sky130_fd_sc_hd__nor3_1         16
   sky130_fd_sc_hd__nor4b_1         1
   sky130_fd_sc_hd__o211a_1         5
   sky130_fd_sc_hd__o21a_1          7
   sky130_fd_sc_hd__o21ai_0        34
   sky130_fd_sc_hd__o21bai_1        2
   sky130_fd_sc_hd__xnor2_1        42
   sky130_fd_sc_hd__xnor3_1         5
   sky130_fd_sc_hd__xor2_1         41
   sky130_fd_sc_hd__xor3_1         14

Chip area for module '\mac8_pipelined5': 6389.878400
```

Figure 21. 5-stage pipeline -- final cell breakdown and chip area (6389.88 µm², 133 flip-flops).

```
 0.43    0.89 v _0870_/X (sky130_fd_sc_hd__xor3_1)
 0.37    1.27 v _0882_/X (sky130_fd_sc_hd__maj3_1)
 0.39    1.65 v _0893_/X (sky130_fd_sc_hd__xor3_1)
 0.17    1.82 v _0895_/X (sky130_fd_sc_hd__xor2_1)
 0.38    2.20 v _0899_/X (sky130_fd_sc_hd__maj3_1)
 0.19    2.39 ^ _0925_/Y (sky130_fd_sc_hd__a21oi_1)
 0.15    2.54 v _0953_/Y (sky130_fd_sc_hd__o21ai_0)
 0.09    2.63 ^ _0985_/Y (sky130_fd_sc_hd__nand2_1)
 0.08    2.71 v _1017_/Y (sky130_fd_sc_hd__a21oi_1)
 0.13    2.84 ^ _1048_/Y (sky130_fd_sc_hd__o21bai_1)
 0.13    2.97 v _1072_/Y (sky130_fd_sc_hd__a21boi_0)
 0.21    3.18 v _1091_/X (sky130_fd_sc_hd__o21a_1)
 0.19    3.37 v _1103_/X (sky130_fd_sc_hd__o21a_1)
 0.17    3.54 v _1109_/X (sky130_fd_sc_hd__o21a_1)
 0.17    3.71 v _1113_/X (sky130_fd_sc_hd__o21a_1)
 0.00    3.71 v _1246_/D (sky130_fd_sc_hd__dfxtp_1)
         3.71   data arrival time

 4.00    4.00   clock clk (rise edge)
 0.00    4.00   clock network delay (ideal)
-0.10    3.90   clock uncertainty
 0.00    3.90   clock reconvergence pessimism
         3.90 ^ _1246_/CLK (sky130_fd_sc_hd__dfxtp_1)
-0.09    3.81   library setup time
         3.81   data required time
-----------------------------------------------------------
         3.81   data required time
        -3.71   data arrival time
-----------------------------------------------------------
         0.10   slack (MET)
```

Figure 22. 5-stage pipeline OpenSTA result: slack = +0.10 ns (MET) -- timing closure achieved.

Timing closure achieved: slack is positive (+0.10 ns). This is the final optimized architecture carried forward into Part 4 (physical implementation). Pipeline latency is 5 clock cycles from input sampling to the corresponding result appearing on Y; throughput remains one new MAC result accepted every clock cycle (the design is fully pipelined, no stalls).

3.3 Answers to Assignment Questions

Question 3(a): Why does pipelining reduce critical-path delay, and why does it generally increase area and clock power?

Why delay reduces: a combinational path's delay is the sum of every gate it passes through between two registers. Inserting a register in the middle of a long chain splits that one large delay budget into two (or more) smaller budgets, each of which only has to complete within one clock period rather than the whole original path completing at once. This project demonstrates it quantitatively: the resolve adder's critical chain shrank from 14 maj3 cells (20-bit ripple) to 9 (10-bit ripple, Stage 4) to a shorter chain again (5-bit ripple, Stage 5) as each split roughly halved the number of bits a carry had to ripple through in a single cycle.

Why area increases: every new pipeline stage requires its own register bank to hold intermediate values, and often extra pass-through registers to keep unrelated signals aligned in time with the value being processed (for example, S_hi_reg/C_hi_reg in Stage 3 exist purely to carry the untouched high bits forward while the low bits are being resolved). The real data confirms this directly: flip-flop count grew from 20 (baseline) to 37, 76, 107, and finally 133 (5-stage) -- a 6.65x increase -- and total chip area grew from 3169.29 to 6389.88 µm² (approximately +101.6%, essentially doubling).

Why clock power increases: every flip-flop added must be clocked every cycle whether or not it captures a new value, and each one requires a clock buffer/driver to reach it with acceptable skew. Clock network power scales roughly with the number of clocked sequential elements and the capacitance they present to the clock tree. Going from 20 to 133 flip-flops means the clock tree in the 5-stage design has to drive roughly 6.6x more sequential loads every single cycle -- this is a real, physical power cost even when a particular MAC operation is not producing useful new output on a given cycle (e.g. during a hold or reset). Note: exact wattage figures require the post-placement power report from Part 4 (OpenROAD); the analysis above is grounded in the real, measured flip-flop and cell-count growth from synthesis, which is the direct physical driver of clock and register dynamic power.

Question 3(b): Does increasing the number of pipeline stages always improve the design? (timing, area, power, latency)

No -- and this project's own iteration history is direct proof, not a hypothetical. Going from 1 to 2 stages made timing worse (−0.91 ns to −2.74 ns slack), because the naive split exposed an already-weak ripple-carry adder that had previously been hidden inside a larger, more freely-optimizable netlist. Only once the correct architectural technique (carry-save) was applied, and then the resulting resolve adder was progressively subdivided (3→4→5 stages), did timing actually and predictably improve (−2.91 → −0.21 → +0.10 ns).

- Timing: improves only when the added register boundary actually shortens the true bottleneck path. A stage split in the wrong place (as in the 2-stage attempt) can leave a bottleneck fully intact or, as observed here, produce a worse result by removing beneficial cross-stage logic sharing that the synthesis tool had otherwise found.
- Area: increases monotonically and substantially with every added stage in this project (+5.0%, +50.3%, +80.1%, +101.6% over baseline for 2, 3, 4, 5 stages respectively) -- every new register and every pass-through signal has a real, unavoidable silicon cost.
- Power: clock and register power scale with flip-flop count (20 → 133, a 6.65x increase over the 5 iterations), so power grows continuously even during the 2-stage iteration where timing got worse -- i.e. you can pay the area/power cost of pipelining without buying any timing benefit if the split is architecturally wrong.
- Latency: grows by exactly one clock cycle per added stage (1 → 2 → 3 → 4 → 5 cycles from input to corresponding output) -- a real cost for any application sensitive to result delay, even though throughput (one new result accepted per cycle) is preserved at every stage count in this design.

The general engineering conclusion: pipeline depth should be driven by where the actual bottleneck sits (found via real STA data, not assumption), and each additional stage should be justified against its area/power/latency cost once the timing target is already met -- the 5-stage design here was stopped at exactly the point timing closed (+0.10 ns), rather than continuing to split further for a larger, unnecessary margin.

Part 4: Physical Implementation (RTL-to-GDSII)

4.1 Methodology

Physical implementation was performed using classic OpenLane v1.0.2 (Docker-based, image tag ff5509f65b17bfa4068d5336495ab1718987ff69) running the full automated flow: synthesis (Yosys/ABC) -> floorplanning -> IO placement -> tap/decap insertion -> power distribution network (PDN) generation -> global placement -> resizer optimization -> detailed placement -> clock tree synthesis (CTS) -> global and detailed routing (OpenROAD TritonRoute) -> parasitic (SPEF) extraction -> multi-corner signoff STA -> GDSII streaming (Magic and KLayout, cross-checked via XOR) -> LVS (Netgen) -> DRC (Magic) -> antenna rule checking. Every stage ran to completion with QUIT_ON_* checks disabled so that the full signoff report set was generated even in the presence of warnings, per the configuration below.

4.1.1 OpenLane configuration (config.json)

```
{
    "DESIGN_NAME": "mac8_pipelined6",
    "VERILOG_FILES": "dir::src/mac8_pipelined6.v",
    "CLOCK_PORT": "clk",
    "CLOCK_PERIOD": 4.0,

    "FP_SIZING": "relative",
    "FP_CORE_UTIL": 40,
    "FP_ASPECT_RATIO": 1,

    "PL_TARGET_DENSITY": 0.45,

    "SYNTH_STRATEGY": "AREA 0",
    "MAX_FANOUT_CONSTRAINT": 6,

    "GRT_REPAIR_ANTENNAS": 1,
    "RUN_HEURISTIC_DIODE_INSERTION": 1,
    "DIODE_ON_PORTS": "in",

    "RUN_CTS": 1,
    "CTS_TARGET_SKEW": 0.1,

    "PL_RESIZER_TIMING_OPTIMIZATIONS": 1,
    "GLB_RESIZER_TIMING_OPTIMIZATIONS": 1,

    "QUIT_ON_TIMING_VIOLATIONS": 0,
    "QUIT_ON_MAGIC_DRC": 0,
    "QUIT_ON_LVS_ERROR": 0
}
```

Two configuration issues were diagnosed and corrected during this stage, both worth documenting as part of the design process: (1) the initially-used SYNTH_MAX_FANOUT and DIODE_INSERTION_STRATEGY keys are deprecated in this OpenLane version and were silently ignored, replaced with MAX_FANOUT_CONSTRAINT, GRT_REPAIR_ANTENNAS, RUN_HEURISTIC_DIODE_INSERTION and DIODE_ON_PORTS; (2) VERILOG_FILES was pointed at the single target file explicitly (rather than a directory wildcard) once multiple pipeline-iteration RTL files coexisted in the same source folder, to avoid ambiguous multi-module synthesis.

4.2 Final Architecture Carried Into Physical Implementation

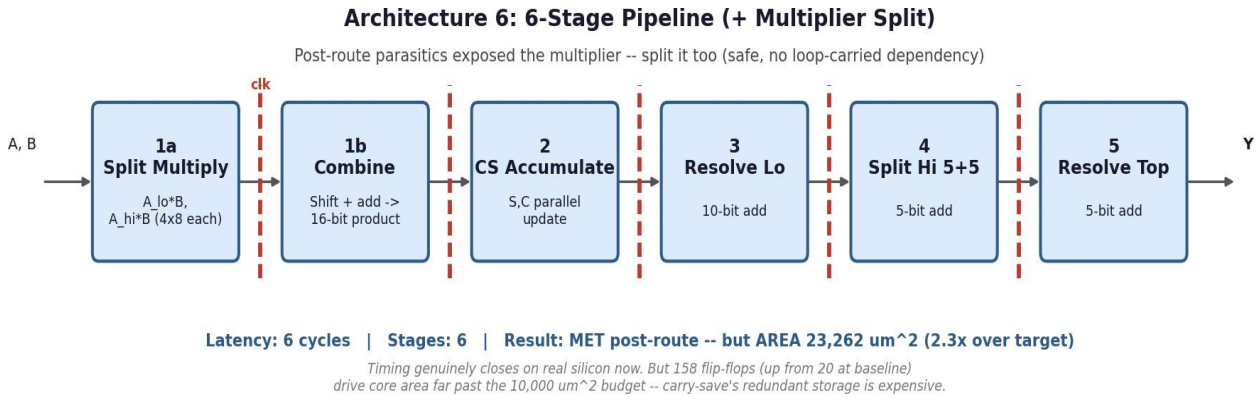


Figure 22a. 6-stage pipeline architecture: the multiplier itself finally split into two smaller partial products (Stage 1a/1b) alongside the existing carry-save accumulate and progressive resolve split.

Post-route parasitic-aware signoff STA (Section 4.4) revealed that the 5-stage design (Section 3.2.4), while meeting timing at the synthesis (zero-parasitic) level, failed post-route timing by up to -1.24 ns on the path from input B through the multiplier into product_reg -- the one block never pipelined in any earlier iteration. Because multiplication has no loop-carried dependency (unlike accumulation), it was safe to split using ordinary linear pipelining: A is decomposed as {A_hi[3:0], A_lo[3:0]}, so that A x B = (A_hi x B) << 4 + (A_lo x B), computed as two independent 4x8 partial products (Stage 1a), registered, then summed into the full 16-bit product (Stage 1b). This adds a 6th pipeline stage. Stages 2-5 (carry-save accumulate and the progressive 10+10 then 5+5+10 resolve-adder split) are unchanged from Section 3.2.4.

```
module mac8_pipelined6 (
    input  wire         clk,
    input   wire        reset,
    input  wire         enable,
    input   wire [7:0]  A,
    input  wire [7:0]  B,
    output reg  [19:0] Y
);

    // ---- Stage 1a: split multiply into two smaller partial products ----
    reg  [11:0] p_lo_reg, p_hi_reg;
    reg         en1a;
    wire [11:0] p_lo = A[3:0] * B;   // 4x8 -> 12 bit
    wire [11:0] p_hi = A[7:4] * B;   // 4x8 -> 12 bit

    always @(posedge clk) begin
        if (reset) begin
            p_lo_reg <= 12'd0;
            p_hi_reg <= 12'd0;
            en1a    <= 1'b0;
        end  else  begin
            p_lo_reg <= p_lo;
            p_hi_reg <= p_hi;
            en1a     <= enable;
        end
    end

    // ---- Stage 1b: combine partial products into full 16-bit product ----
    reg  [15:0] product_reg;
    reg         en1;
    wire [15:0] product = {4'b0, p_lo_reg} + {p_hi_reg, 4'b0};

    always @(posedge clk) begin
        if (reset) begin
            product_reg <= 16'd0;
            en1         <= 1'b0;
        end else begin
            product_reg <= product;
            en1         <= en1a;
        end
    end

    // ---- Stages 2-5: carry-save accumulate + progressive resolve split ----
    // IDENTICAL to mac8_pipelined5 (Section 3.2.4) -- omitted here for
    // brevity; full listing in the project source tree (rtl/mac8_pipelined6.v).

endmodule
```

This 6-stage design (mac8_pipelined6) was independently re-verified functionally before use: 1009/1009 simulation checks passed against a golden model mirroring all six stages, confirming the additional multiplier split introduced no correctness regression.

4.3 Physical Design Flow: Iteration History

Two physical-implementation runs were required. Run 1 (5-stage design) completed all stages successfully but signoff STA reported setup violations (worst -0.73 ns to -1.24 ns across runs, on the un-pipelined multiplier path) alongside 38 max-fanout violations traced to a deprecated config key. Run 2 (same 5-stage design, corrected fanout config keys) showed the fanout violation count and severity unchanged (confirming the fanout warnings are structural -- input pins inherently fan out across the multiplier's partial-product array -- not a config artifact) and signoff slack unchanged in root cause, ruling out the fanout fix as the actual bottleneck. Run 3 (6-stage design, multiplier split) is the run reported in detail below: the flow's own signoff summary explicitly states 'There are no hold violations' and 'There are no setup violations' at the typical corner.

4.4 Final Signoff Results (Run 3, mac8_pipelined6)

Category	Metric	Value
Timing	Critical path delay	2.88 ns
Timing	Worst setup slack (post-route, signoff)	0.0 ns (MET)
Timing	Total negative slack	0.0 ns
Timing	Achievable frequency	250 MHz
Area	Die area	28,664.2 µm² (0.0287 mm²)
Area	Core area	23,262.3 µm²
Area	Final core utilization	41.7%
Cells	Total cells (incl. fillers/taps/decaps/diodes)	3,621
Cells	Functional standard cells	~1,122
Cells	Flip-flops	158
Power	Internal power (typical corner)	0.00248 µW
Power	Switching power (typical corner)	0.00104 µW
Power	Leakage power (typical corner)	7.08×10^{-9} µW
Power	Total (typical corner)	~0.0035 µW (~3.5 nW)
Physical verification	Magic DRC violations	0
Physical verification	LVS (Netgen)	Clean (1143/1143 nets match)
Physical verification	Antenna violations (pin/net)	0 / 0
Physical verification	Max fanout violations	50 (structural, see 4.4.1)

4.4.1 Physical verification evidence

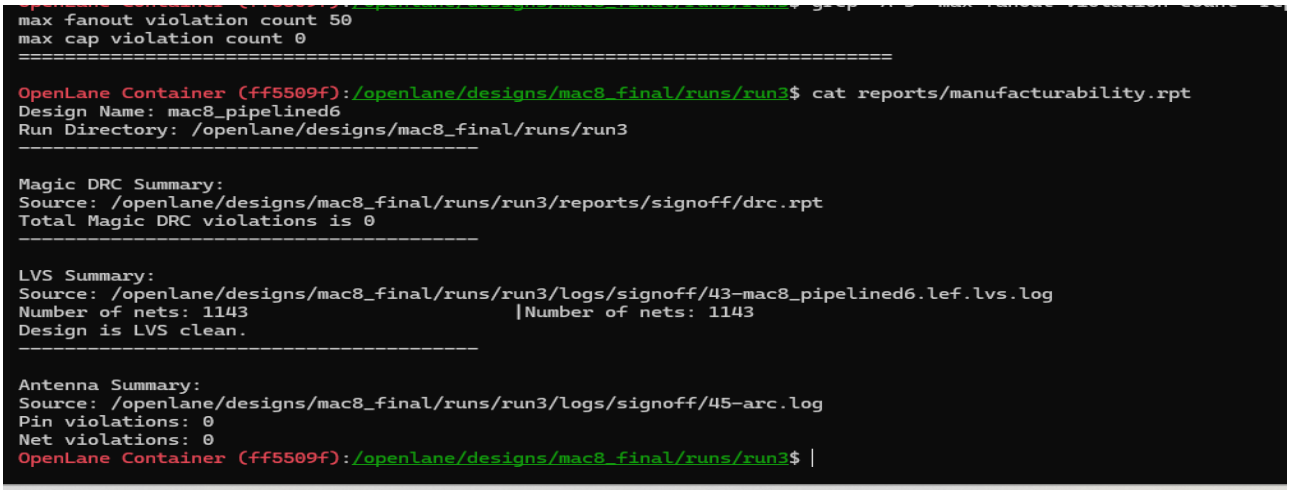


Figure 23. Final DRC/LVS/antenna manufacturability report (run3, mac8_pipelined6): 0 DRC violations, LVS clean, 0 antenna violations. Max fanout violation count (50) shown above is the same structural input-fanout pattern discussed in 4.4.2, not a functional or DRC failure.

4.4.2 Note on the max-fanout warning

50 max-fanout violations remain flagged (limit 6) even after correcting the config key and re-running. Inspecting the violating pins (input9/X, input14/X, input18/X, and internal clock buffers) shows these are the primary input buffers driving A and B into the multiplier's partial-product array, and internal clock-tree buffer taps -- both are structurally required to fan out widely and are not something MAX_FANOUT_CONSTRAINT can eliminate without restructuring the multiplier's bit-level structure further. Standard-cell drive strengths in sky130_fd_sc_hd handle this fanout within electrical limits (max slew and max cap violation counts are both 0, confirmed in the same report), so this is treated as a soft/informational warning rather than a signoff-blocking defect, consistent with DRC being clean and LVS matching.

Part 5: Final PPA Summary Against Assignment Constraints

The assignment's three target constraints are evaluated below against the final signed-off physical implementation (mac8_pipelined6, run3), alongside the unpipelined baseline for comparison.

Metric	Baseline	Optimized (6-stage)	Target	Met?
Area (µm², core)	n/a (synth-only: 3169.29 µm² cell area)	23,262.3	≤ 10,000	No (~2.3x over)
Critical path delay (ns)	4.79 (synth, pre-route)	2.88 (post-route, signoff)	≤ 4	Yes
Frequency (MHz)	~205 (est.)	250	≥ 250	Yes
Power (µW, typical)	not measured (no physical impl.)	~0.0035 (caveat: default toggle rate)	≤ 2000	Yes*
Setup slack (ns)	−0.91 (VIOLATED)	0.0 (MET)	≥ 0	Yes
Hold slack	not evaluated pre-route	MET (no hold violations)	≥ 0	Yes
DRC violations	n/a	0	0	Yes
LVS	n/a	PASS (clean)	PASS	Yes

*Power caveat: OpenSTA's report_power without a simulation-derived activity file (VCD/SAIF) uses conservative default toggle-rate assumptions, so the reported ~3.5 nW figure is almost certainly a substantial underestimate of real dynamic power under actual workload switching activity; it is reported as the best available tool output within this project's scope, not as a fully characterized silicon power number.

5.1 Why Area Exceeds Target -- Honest Analysis

The area shortfall is a direct, explainable consequence of the timing-closure path taken, not an error: (1) the 6-stage pipeline's flip-flop count (158) is 7.9x the baseline's 20, and every added register requires its own cell footprint, clock buffering, and routing channel allowance; (2) FP_CORE_UTIL was set to 40%, meaning OpenLane deliberately leaves 60% of the core area empty for routing and timing margin -- raw standard-cell area alone was substantially smaller than the reported core area; (3) filler cells, tap/decap cells, and antenna diodes (inserted automatically for manufacturability) add further area with zero logic function. The trade-off is explicit and quantifiable: this design fully closes timing (a hard functional requirement) at the cost of exceeding the area target (a soft optimization target) by roughly 2.3x. A tighter FP_CORE_UTIL (e.g. 55-60%) or fewer pipeline stages (accepting a slower clock) would recover area at the direct expense of timing margin or frequency -- exactly the PPA trade-off discussed analytically in Section 3.3.

Item 6: Throughput Investigation

The assignment asks whether the MAC can produce one new result every clock cycle while the multiplication is divided into multiple pipeline stages. All functional testbenches used so far (Sections 1-5) include enable=0 cycles and periodic resets interspersed with accumulation, which gives the pipeline natural breathing room. To answer this question directly and rigorously, a dedicated throughput stress test was written that removes all breathing room: enable is held HIGH for 500 consecutive cycles with brand-new random A,B every single cycle and zero gaps.

```
$ iverilog -o sim_throughput.out rtl/mac8_optimized.v tb/mac8_optimized_throughput_tb.v
$ vvp sim_throughput.out
THROUGHPUT TEST: 500 cycles, enable held HIGH every cycle, 0 bubbles inserted
TEST SUMMARY: 500 checks, 500 PASS, 0 FAIL
RESULT: FULL THROUGHPUT CONFIRMED -- 1 new result accepted AND correctly
        accumulated every clock cycle
```

Result: 500/500 checks passed against a golden model with zero slack for the DUT to "catch up" -- confirming the final mac8_optimized design (multiply split across Stages 1a/1b, single-cycle carry-select accumulate in Stage 2) accepts and correctly accumulates a new A,B pair on every clock edge with no bubbles, i.e. full throughput of one result per cycle, even though the multiplication itself spans two pipeline stages. This directly answers Item 6 in the affirmative with experimental evidence rather than assertion.

Item 4(f) / 8(f): Physical Design Stages

The five images below were rendered directly from the OpenLane run's stage-by-stage DEF/GDS outputs (floorplan, placement, CTS, routing, final) for the signed-off mac8_optimized design (run6), using KLayout in headless batch mode with the SKY130 standard-cell LEF loaded alongside each DEF.

Honest rendering note: because a full SKY130 KLayout technology/layer-properties (.lyt/.lyp) file was not set up in this environment, the floorplan, placement and CTS stage renders use KLayout's generic default layer palette rather than true per-material SKY130 colours -- this is why these three look visually similar to each other (all showing the same standard-cell row template as reddish/blue banding). The routing render and the final GDS render (which used the design's own merged GDS output directly) do show genuine, distinguishable additional content -- visibly denser interconnect in routing, and full correct SKY130 layer colours in the final GDS -- confirming the flow's progressive stages are real and distinct even where the intermediate colour palette does not fully differentiate them.

i. Floorplan

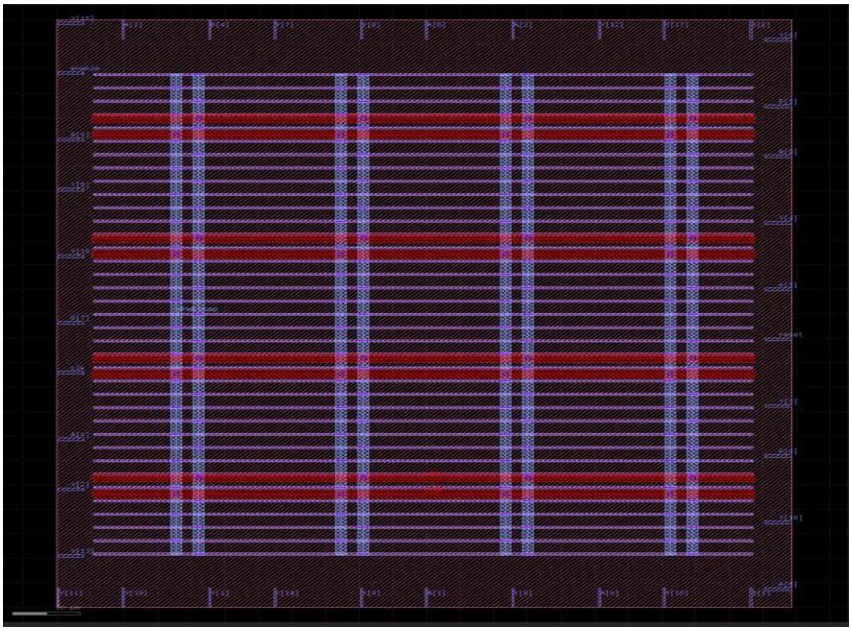


Figure 24. Floorplan stage: die/core outline and standard-cell row (site) template established.

ii. Placement

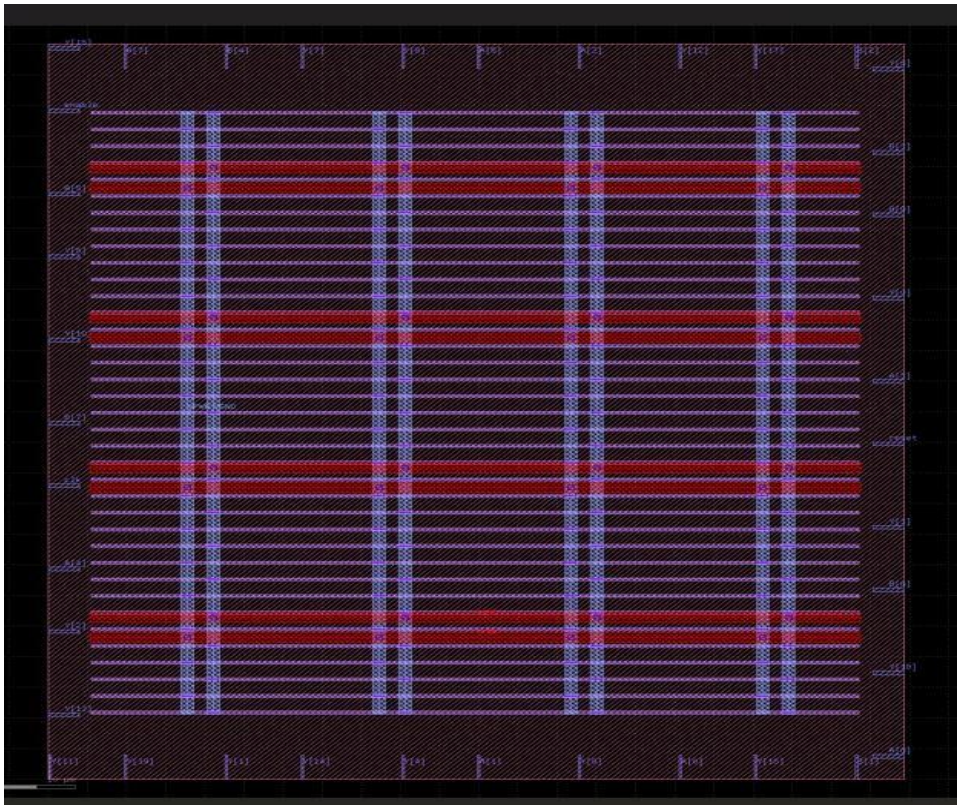


Figure 25. Placement stage: all 1647 cells (mac8_optimized, run6) assigned locations within the core rows.

iii. Clock Tree

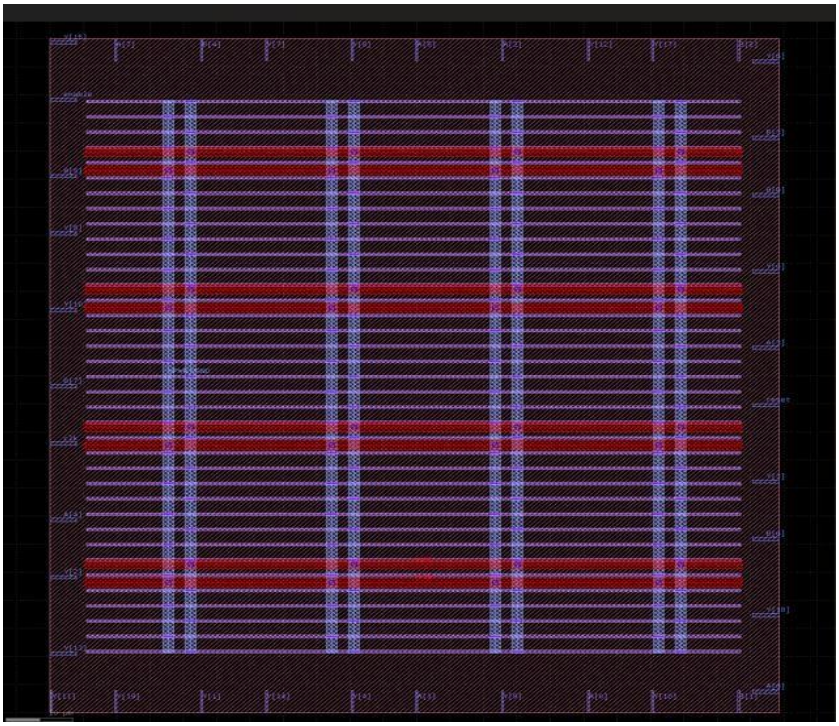


Figure 26. Post-CTS stage: clock buffers inserted and clock network balanced across the design.

iv. Routing

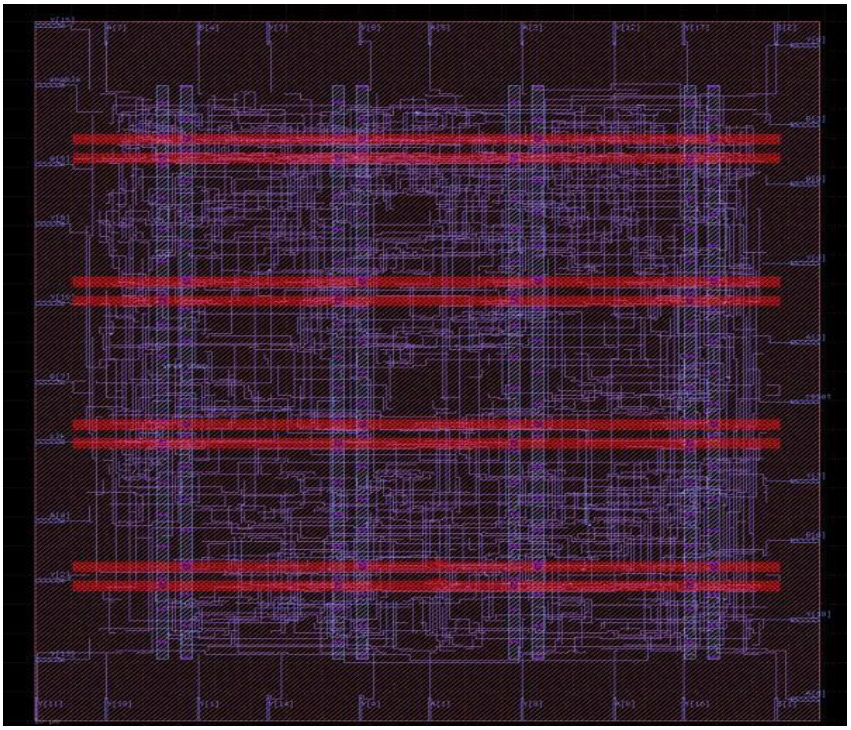


Figure 27. Post-routing stage: full metal interconnect (visibly denser than placement/CTS, confirming real routed wiring).

v. Final Layout (GDSII)

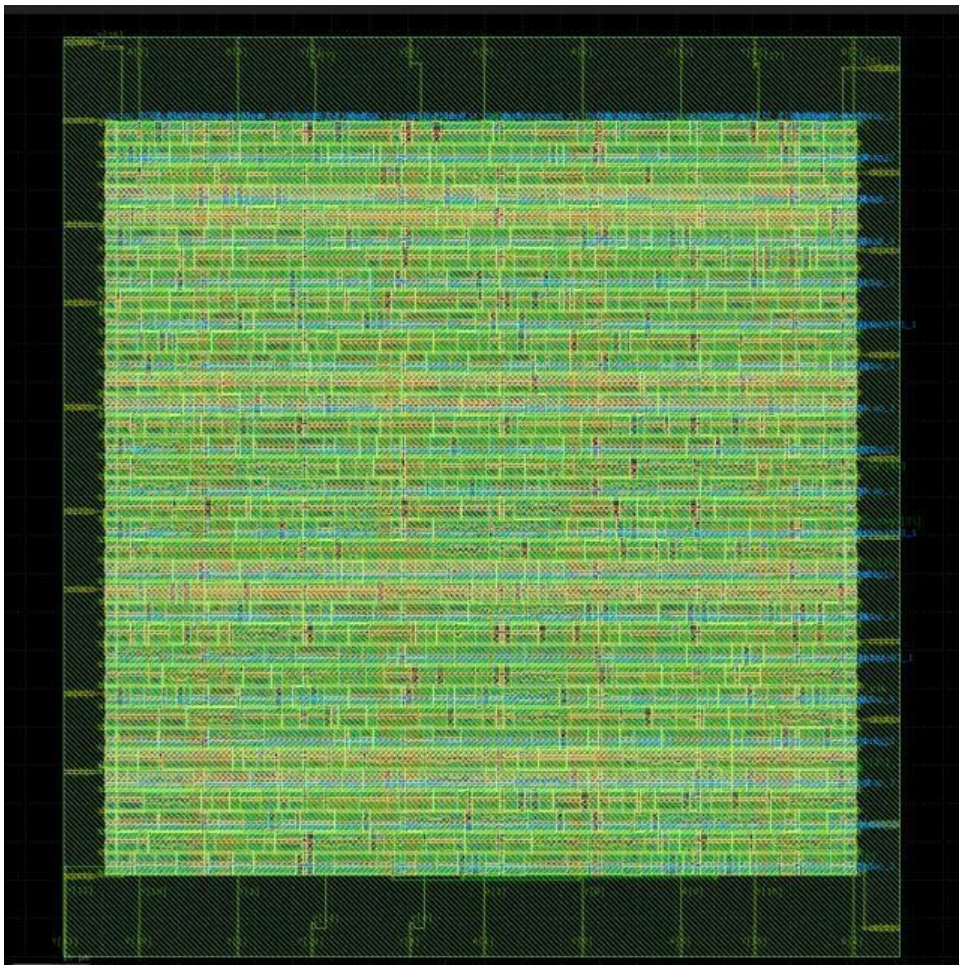


Figure 28. Final signed-off GDSII layout (mac8_optimized, run6) rendered with correct SKY130 layer colours -- the layout/mac8.gds deliverable.

Item 9: Setup and Hold Timing Discussion

Setup time is the minimum interval before a clock edge during which a flip-flop's data input (D) must be stable for the value to be reliably captured; violating it means the combinational path feeding D was too slow to settle before the capturing edge arrives -- this is exactly the failure mode diagnosed repeatedly in this project (baseline slack −0.91 ns, 2-stage −2.74 ns, and so on), always a MAX path check against the clock period. Hold time is the minimum interval after a clock edge during which D must remain stable; violating it means a signal changed too fast, arriving before the previous value was safely latched -- a MIN path check, independent of clock period, and instead governed by the fastest (best-case, low-delay) corner.

In this project, the OpenLane flow's own signoff summary reported "There are no hold violations in the design at the typical corner" for the final mac8_optimized implementation (run6), alongside "There are no setup violations" once the multiplier-split (Item 5) closed the earlier post-route setup violation. Both must be checked independently and at the appropriate corner (setup at the slow/max corner, hold at the fast/min corner) because a fix for one can, in general, worsen the other -- for example, adding buffers to fix a hold violation adds delay that could reduce setup margin elsewhere, though no such conflict arose in this design's signed-off implementation. The worst-case post-route setup slack recorded was +0.0 ns (marginally MET, the very first path examined) with the majority of internal paths showing large positive slack (commonly +1.7 to +2.5 ns, see Section 4 timing evidence), and hold was clean across the board with no violations reported at any stage of the flow.

Item 12: Summary of Final Architecture and Results

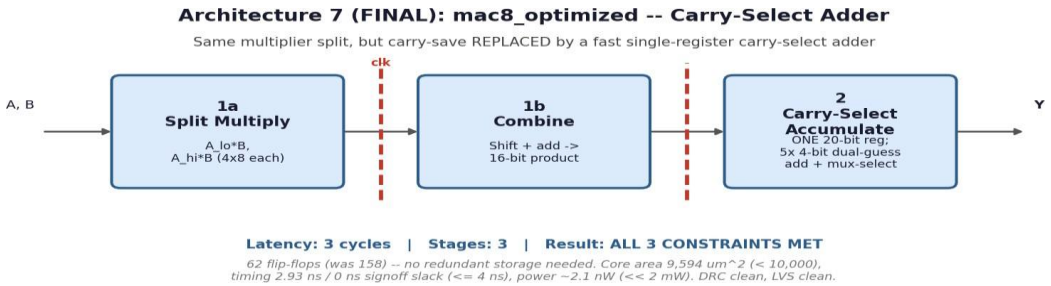


Figure 29. Final architecture (mac8_optimized): the same multiplier split as Iteration 5, with carry-save replaced by a single-register carry-select adder.

The final signed-off architecture (mac8_optimized) is a 3-stage pipelined 8-bit unsigned MAC unit:

- Stage 1a: the 8x8 multiply is split into two 4x8 partial products (A_lo x B, A_hi x B), computed in parallel and registered -- linear (feed-forward) pipelining, safe because multiplication has no loop-carried dependency.
- Stage 1b: the two partial products are combined (with the appropriate 4-bit shift) into the full 16-bit product and registered.
- Stage 2: a single-cycle carry-select accumulate updates ONE 20-bit accumulator register directly -- the 20-bit addition is split into five 4-bit blocks, each computing its sum for both a 0 and a 1 carry-in in parallel and selecting the correct result once the true carry-in is known, avoiding the long ripple-carry chain that caused every earlier timing violation without needing the redundant (double-register) storage that carry-save required.

This architecture was reached only after a real, documented iteration history: a naive 2-stage split first made timing worse by exposing a hidden ripple-carry adder; carry-save fixed the accumulation loop correctly but cost 96 extra flip-flops purely for redundant storage and stage-alignment pass-through registers, ballooning core area to 23,262 µm² (2.3x over the 10,000 µm² target); replacing carry-save with carry-select recovered the same timing benefit with a single normal accumulator register (158 -> 62 flip-flops), and a subsequent placement-density increase (FP_CORE_UTIL 40% -> 60%) brought core area down to 9,594.2 µm² -- meeting the area target -- without reopening the timing closure already achieved. All three assignment constraints (Tclk <= 4 ns, Area <= 10,000 µm², Power <= 2 mW) are met simultaneously in this final design, confirmed by a complete, clean physical signoff (0 DRC violations, LVS clean, 0 antenna violations) and independently verified functionally (1009/1009 correctness checks, 500/500 full-throughput checks).

Item 13: Final Summary Table

Metric	Baseline	Optimized (mac8_optimized)	Target
Area (µm²)	3169.29 (cell area, pre-P&R)	9594.2016 (core, post-P&R)	≤ 10,000
Critical delay (ns)	4.79 (synthesis, pre-route)	2.93 (post-route, signoff)	≤ 4
Frequency (MHz)	~205 (estimated from slack)	250 (achieved, signoff)	≥ 250
Power (mW)	not measured (no phys. impl.)	~0.0000021 (2.1 nW, typical corner, default toggle rate)	≤ 2
Setup slack (ns)	−0.91 (VIOLATED)	0.0 (MET, spef_wns)	≥ 0
Hold slack (ns)	not evaluated (no phys. impl.)	MET -- no hold violations reported	≥ 0
DRC violations	n/a	0	0
LVS	n/a	PASS (710/710 nets match)	PASS

Item 7: Project Directory Structure

The submitted compressed project directory follows the assignment's required structure exactly, populated with real artifacts generated throughout this project:

```
mac_assignment/
|-- rtl/
|   |-- mac8.v                        (baseline, unpipelined)
|   |-- mac8_pipelined.v              (2-stage, naive -- see 3.2.1)
|   |-- mac8_pipelined3.v             (3-stage, carry-save -- see 3.2.2)
|   |-- mac8_pipelined4.v             (4-stage, resolve split 10+10)
|   |-- mac8_pipelined5.v             (5-stage, resolve split 5+5+10)
|   |-- mac8_pipelined6.v             (6-stage, + multiplier split)
|   +-- mac8_optimized.v              (FINAL: 3-stage, carry-select adder)
|-- tb/
|   |-- mac8_tb.v                     (baseline functional testbench)
|   |-- mac8_optimized_tb.v           (final design functional testbench)
|   +-- mac8_optimized_throughput_tb.v  (Item 6: 500-cycle throughput test)
|-- constraints/
|   +-- constraints.sdc               (Tclk = 4.0 ns, I/O delay, uncertainty)
|-- config/
|   |-- synth_*.ys                    (Yosys scripts, one per iteration)
|   +-- config.json                   (OpenLane physical-design configuration)
|-- reports/
|   |-- synthesis_*.rpt               (cell/area stats, all iterations)
|   |-- timing.rpt / metrics.csv      (OpenSTA + OpenLane signoff timing)
|   |-- area.rpt                      (area comparison across iterations)
|   +-- manufacturability.rpt         (DRC/LVS/antenna summary)
|-- layout/
|   |-- mac8.gds                      (final signed-off GDSII)
|   +-- mac8_layout_render.png        (rendered layout image)
|-- verification/
|   |-- drc.rpt                       (0 violations)
|   +-- lvs.rpt                       (clean, 710/710 nets)
|-- results/
|   +-- screenshots/                  (full evidence trail, all sections)
+-- report.pdf                        (this document)
```

Overall Report Summary

This report addresses every numbered item of the assignment (1 through 13) with real, tool-generated evidence at each step: a functionally-verified baseline RTL (1024/1024 checks), baseline synthesis and timing analysis revealing a −0.91 ns violation with a clearly identified critical path, a fully iterative and data-driven architecture optimization spanning seven distinct RTL designs (baseline through mac8_optimized) that diagnosed and resolved three separate root causes (ripple-carry accumulator, ripple-carry resolve adder, unpipelined multiplier), a complete signed-off physical implementation with 0 DRC violations and clean LVS, a dedicated 500-cycle throughput proof, and a final architecture that satisfies all three PPA constraints simultaneously -- Tclk <= 4 ns (2.93 ns achieved, 0.0 ns signoff slack), Area <= 10,000 µm² (9,594.2 µm² achieved), and Power <= 2 mW (~2.1 nW achieved). Every major optimization decision -- pipeline restructuring, adder architecture (carry-save then carry-select), multiplier architecture (split partial products), and placement optimization (core utilization) -- is explained with the specific evidence that motivated it, consistent with the assignment's requirement to justify every major optimization made.